BioGuard

Low-Cost Biogas Level and Leak Monitor for Household Digesters

A Report Submitted in
Partial Fulfilment for
award of Bachelor of Technology

In

Design Thinking Project (Problem Statement 24)

COMPUTER SCIENCE & ENGINEERING IN ARTIFICIAL INTELLIGENCE

By

Anjaneya Mallick (2401331520036)

Anurag Singh (2401331520045)

Ayush Raj (2401331520060)

Under the Supervision of

Ms. Barkha Bhardwaj

Assistant Professor



Department of Computer Science & Engineering in Artificial Intelligence

School of Computer Science and Information Technology

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY

GREATER NOIDA

(An Autonomous Institute)

Affiliated to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW

DECLARATION

We hereby declare that the work presented in this report was carried out by us. We have not submitted the matter embodied in this report for the award of any degree or diploma of any other University or Institute.

Team Leader:
Anjaneya Mallick (Roll No.: 2401331520036)

_____

(Candidate's Signature)

Team Member:
Anurag Singh (Roll No.: 2401331520045)

_____

(Candidate's Signature)

Team Member:
Ayush Raj (Roll no.:2401331520060)

(Candidate's Signature)

CERTIFICATE

Certified that **Anjaneya Mallick (2401331520036)**, **Anurag Singh (2401331520045) and Ayush Raj (2401331520060)** have carried out the project work presented in this Project Report in partial fulfillment of the requirements for the award of **Bachelor of Technology CSE(AI) 3rd Year** from **Noida Institute of Engineering & Technology** under my supervision.

Signature: _____

Ms. Barkha Bhardwaj
Assistant Professor
Dept of CSE
NIET, Gr. Noida

Signature: _____

Dr. Anand Kumar Gupta
HOD of CSE-AI
NIET, Gr. Noida

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to **Ms. Barkha Bhardwaj** for her invaluable guidance, support, and constant supervision throughout the development of this project. Her expertise and encouragement have been pivotal in shaping the project.

We would also like to extend our thanks and appreciation to our esteemed **HOD (Dr. Anand Kumar Gupta)** for the continuous motivation and support. The encouragement received has been instrumental in completing this project successfully.

Finally, we are deeply thankful to our peers and everyone who directly or indirectly contributed to this project with their knowledge, advice, and assistance.

ABSTRACT

Households that cook with small biogas digesters usually have no simple way to see how much gas is stored, and a slow leak at a pipe joint can go unnoticed until the cooking flame weakens. This wastes fuel, interrupts cooking and creates a safety risk. BioGuard is a design-thinking project that addresses this problem (Problem Statement 24, Bioenergy / Environmental Biotechnology) by proposing a compact unit that shows the available gas level and warns early about a possible leak.

The proposed physical system uses a low-pressure sensor at the digester outlet to estimate stored gas, an MQ-series combustible-gas sensor near the pipe joints, an ESP32 or Arduino controller that compares readings with thresholds, an LED bar-graph for level display and a buzzer for leak alarms. Because physical hardware was not available within the project timeline, the implemented deliverable is the BioGuard Console, a self-contained browser-based simulation developed using HTML, CSS and JavaScript.

The console demonstrates a gas-availability index with GOOD, LOW and CRITICAL states, a virtual outlet-pressure channel, a possible-leak alert, a 10-reading trend chart, a 30-event log, editable thresholds, selectable alert modes, Web Audio alarm tones and four visual themes. Twelve scenario tests were executed on the prototype and all matched the expected behaviour.

The prototype is a software simulation only. Its pressure (kPa*) and gas-sensor (ppm*) values are illustrative, the gas index is not a measured volume and the system is not a certified gas-safety device. The report therefore also documents the limitations and the roadmap towards sensor integration, calibration and supervised validation.

TABLE OF CONTENTS

	Page No.
Declaration	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
List of Figures and Tables	vi
CHAPTER 1: INTRODUCTION	**1**
1.1 Overview	1
1.2 Problem Statement	1
1.3 Objective and Scope	2
CHAPTER 2: LITERATURE REVIEW	**3**
2.1 Overview of Existing Systems	3
2.2 Limitations of Existing Systems	3
CHAPTER 3: DESIGN THINKING METHODOLOGY	**5**
3.1 Empathize	5
3.2 Define	5
3.3 Ideate	6
3.4 Prototype	6
3.5 Test	6
CHAPTER 4: REQUIREMENTS	**7**
4.1 Functional Requirements	7
4.2 Non-Functional Requirements	7
4.3 Software and Hardware Requirements	8
CHAPTER 5: SYSTEM DESIGN AND IMPLEMENTATION	**9**
5.1 System Architecture	9
5.2 Alert Decision Logic	9
5.3 GUI Design	10
5.4 Data Design	12
5.5 Implementation Details	13
CHAPTER 6: TESTING AND RESULTS	**15**
6.1 Testing Methodology	15
6.2 Test Cases and Results	15
6.3 Output Screens	16
CHAPTER 7: FEASIBILITY, IMPACT AND SAFETY	**19**
7.1 Feasibility Analysis	19
7.2 Impact and Benefits	19
7.3 Safety Considerations	19
CHAPTER 8: CONCLUSION AND FUTURE WORK	**20**
8.1 Limitations	20
8.2 Conclusion	20
8.3 Future Work	20
APPENDIX	**21**
REFERENCES	**23**

LIST OF FIGURES

Figure 3.1: Design thinking process followed in BioGuard 5
Figure 5.1: System architecture of BioGuard 9
Figure 5.2: Alert decision logic of the BioGuard Console 10
Figure 5.3: BioGuard Console home view in normal state (gas 72% GOOD, leak CLEAR) 11
Figure 5.4: Gas reserve profile and recent trend of the last 10 readings 11
Figure 5.5: Control room (editable parameters) and alarm and safety controls 11
Figure 5.6: Sensor channels (illustrative only) with prototype-boundary notice 12
Figure 5.7: Event log of recent actions 12
Figure 5.8: Software modules of the BioGuard Console 13
Figure 6.1: Output screen: normal operation (gas 72% GOOD, leak CLEAR) 16
Figure 6.2: Output screen: low-gas warning (gas 24% LOW, amber cue, system LOW GAS) 17
Figure 6.3: Output screen: critical gas level (gas 9% CRITICAL, red state, alert active) 17
Figure 6.4: Output screen: possible-leak alert (leak ALERT, alarm ON, system ALERT) 18

LIST OF TABLES

Table 1.1: Project details 2
Table 2.1: Summary of sources reviewed 3
Table 2.2: Identified gaps and the BioGuard response 4
Table 3.1: User needs and pain points 5
Table 3.2: Ideas considered and decision 6
Table 4.1: Functional requirements 7
Table 4.2: Non-functional requirements 7
Table 4.3: Proposed hardware modules (not connected in the current prototype) 8
Table 5.1: Alert states and conditions (demo defaults) 10
Table 5.2: Dashboard components 12
Table 5.3: Main state variables of the console 13
Table 5.4: Key functions of the prototype 14
Table 6.1: Test cases and results 15
Table 6.2: Summary of output states 18
Table 7.1: Feasibility analysis 19
Table 7.2: Potential impact and benefits 19

CHAPTER 1: INTRODUCTION

1.1 Overview

Biogas is a renewable fuel produced when organic material such as animal dung and kitchen waste is digested in the absence of oxygen. Small family-type digesters are widely used in rural and semi-urban India as a source of clean cooking fuel, and the Ministry of New and Renewable Energy (MNRE) reports about 4.31 million family-type biogas plants set up in the country.

A household digester stores gas but offers no easy indication of how much is left. Users usually discover that the reserve is low only when the cooking flame turns weak, and a slow leak at a pipe joint may continue unnoticed. This wastes fuel, interrupts cooking and is a safety concern because biogas is combustible.

BioGuard is a design-thinking project that proposes one compact monitoring unit for both needs. A low-pressure sensor at the digester outlet would estimate the stored gas and show it on an LED bar-graph, while an MQ-series gas sensor near the pipe joints would trigger a buzzer when a possible leak is detected. An ESP32 or Arduino controller compares the readings with editable thresholds.

Physical hardware was not available within the project timeline. The implemented deliverable is therefore the **BioGuard Console**, a self-contained browser-based simulation built with HTML, CSS and JavaScript. It demonstrates the intended interface, alert thresholds, alarm behaviour and event history using illustrative values. This report documents the problem, the design-thinking process, the software implementation, testing, limitations and the roadmap towards a physical prototype.

1.2 Problem Statement

Problem Statement 24: Low-Cost Biogas Level and Leak Monitor for Household Digesters (Domain: Bioenergy / Environmental Biotechnology). The statement is taken from the Cross-Domain Design Thinking Problem Statement Compendium.

Households using small biogas digesters for cooking may not know how much gas is stored or notice a slow leak until the cooking flame weakens unexpectedly. This can waste fuel and pose a safety risk.

Design question: How might we give a household a simple, visible readout of how much biogas is available, and an early warning if gas is leaking?

Table 1.1: *Project details*

Item	Details
Project title	BioGuard: Low-Cost Biogas Level and Leak Monitor for Household Digesters
Project ID and category	24 | Bioenergy / Environmental Biotechnology
Team leader	Anjaneya Mallick (Roll No. 2401331520036)
Team member	Anurag Singh (Roll No. 2401331520045)
Guide	Ms. Barkha Bhardwaj, Assistant Professor, Dept of CSE
Programme	B.Tech CSE (Artificial Intelligence), 3rd Year, NIET Greater Noida
Deliverable built	BioGuard Console: browser-based simulation (BioGuard_Demo.html)

1.3 Objective and Scope

Objective

The main objective of this project is to design a household biogas monitoring concept using the design-thinking approach and to build a working software prototype that shows gas availability, low-gas and critical states, and a possible-leak warning in an easy-to-understand interface.

Additional objectives include:

- To understand the needs of households that use small biogas digesters
- To present gas availability as a simple index with GOOD, LOW and CRITICAL states and a 10-segment LED-style bar
- To demonstrate a possible-leak alert with visual and audible warnings
- To provide editable thresholds, alert modes and alarm settings
- To keep a short trend and event history of recent readings
- To outline a low-cost route towards a physical sensor-based prototype

Scope

The target users are rural and semi-urban homes that cook with 1 to 6 m3 family-type biogas plants. The concept can later be extended into a physical unit that combines a pressure sensor, a combustible-gas sensor, a controller, an LED bar-graph and a buzzer, with an optional mobile alert.

The scope of the work completed in this report is the software simulation. It covers the interface, the alert logic and the alarm behaviour. It does not include physical sensing, calibration, cloud connectivity or any certified safety function; these are listed as future work in Chapter 8.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of Existing Systems

The BioGuard concept draws on the biogas programme in India, the design-thinking method, low-cost gas-sensing modules and common microcontroller platforms. Table 2.1 summarises the sources reviewed and how each one informed the project.

Biogas in India. MNRE documents the national family-type biogas programme and reports about 4.31 million family-type plants. These plants are the target setting for BioGuard because they serve households that cook daily with stored gas.

Design thinking. The Stanford d.school Design Thinking Bootleg describes five modes of work: Empathize, Define, Ideate, Prototype and Test. BioGuard follows these modes in order and uses the Test results to refine the prototype.

Gas sensing. The Hanwei MQ-4 datasheet describes a semiconductor sensor for natural gas and methane. MQ-series modules are inexpensive and widely available, which makes them suitable for a low-cost leak indicator. Their readings must be calibrated and validated before they are trusted for safety decisions.

Controller platform. The ESP32 and Arduino families are common, low-cost controllers. The ESP32 additionally supports Wi-Fi, which makes the optional mobile alert possible in a later stage.

Table 2.1: *Summary of sources reviewed*

No.	Source	Content used	Role in BioGuard
1	Problem Statement 24, Design Thinking Compendium	Low-cost biogas level and leak monitor for household digesters	Problem definition
2	Hanwei Electronics, MQ-4 datasheet	Semiconductor sensor for natural gas / methane	Leak-sensing module
3	MNRE, Government of India, Biogas FAQs	Family-type biogas plants and national programme	Context and target users
4	Stanford d.school, Design Thinking Bootleg	Empathize, Define, Ideate, Prototype, Test	Project methodology
5	Espressif Systems, ESP32 documentation	Microcontroller with Wi-Fi support	Proposed controller
6	MDN Web Docs	HTML, CSS, JavaScript, SVG, Web Audio API, localStorage	Prototype implementation

2.2 Limitations of Existing Systems

The problem statement and the reviewed approaches point to several gaps that BioGuard tries to address:

- **No visible gas level.** Many households have no visible indicator of stored gas and notice a problem only when the flame weakens.
- **Late leak awareness.** A slow leak at a joint can continue for a long time before it is noticed, wasting fuel and adding a safety risk.

- **Separate, single-purpose devices.** Level indication and leak detection are usually treated as different problems, so a household would need more than one device.
- **Dependence on phones or internet.** A solution that works only through a smartphone excludes homes with limited connectivity. A local LED bar and buzzer work without a phone.
- **Fixed behaviour.** Fixed alert limits do not suit every digester size or household. Editable thresholds and alert modes give the user control.
- **Calibration need.** Pressure alone does not give an accurate gas volume, and gas-sensor thresholds need controlled validation. Any physical version must therefore be calibrated for the particular digester.

These limitations show the need for a simple, low-cost and locally working monitor. Table 2.2 maps each gap to the response built into BioGuard, and the gaps shaped the requirements in Chapter 4.

Table 2.2: *Identified gaps and the BioGuard response*

Gap	BioGuard response
No visible gas level	Gas-availability index with gauge, 10-segment LED-style bar and GOOD / LOW / CRITICAL states
Late leak awareness	Leak channel with threshold alert, red warning banner and alarm tone
Separate devices	One console combining level and leak monitoring
Dependence on phones or internet	Local LED bar and buzzer proposed; prototype runs offline in a browser
Fixed behaviour	Editable thresholds, four alert modes, volume and sound-pattern settings
Calibration need	Index and ppm* / kPa* values clearly marked illustrative; calibration listed as future work

CHAPTER 3: DESIGN THINKING METHODOLOGY

BioGuard follows the five modes of design thinking. Figure 3.1 shows the sequence and the main outcome of each stage; the following sections describe each stage in detail.

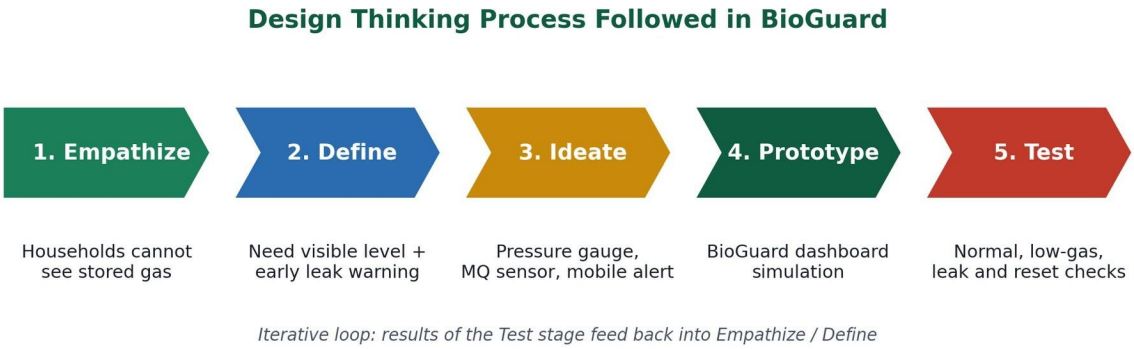


Figure 3.1: *Design thinking process followed in BioGuard*

3.1 Empathize

The target users are households that cook with small biogas digesters. The pain points described in the problem statement are uncertainty about how much gas is stored and late awareness of slow leaks. Table 3.1 summarises what these users need and what troubles them.

Table 3.1: *User needs and pain points*

User	Need	Pain point
Cook at home	Know whether enough gas remains before starting to cook	Flame weakens suddenly in the middle of cooking
Household member	Be told early if gas is escaping near the pipe joints	Slow leaks are not noticed; fuel is wasted and risk increases
Digester owner	Simple, low-cost unit that works without a smartphone	No visible level indicator or alarm on the plant
Family on limited internet	Local indication using lights and a buzzer	App-based solutions need connectivity

3.2 Define

From the observations above, the design challenge is to provide a simple visible readout of the available biogas and an early warning when a possible leak is detected. The point of view of the project is:

> *"Households using small biogas digesters need a simple, visible way to know how much gas is available and to be warned early about a leak, because they currently find out only when the flame weakens."*

This is captured in the design question: **How might we give a household a simple, visible readout of how much biogas is available, and an early warning if gas is leaking?**

3.3 Ideate

Several ideas were considered to answer the design question. Table 3.2 lists them and the decision taken for the prototype.

Table 3.2: *Ideas considered and decision*

Idea	Description	Decision
Pressure-based gas gauge	Low-pressure sensor at the digester outlet gives a stored-gas level shown on an LED bar-graph	Selected (level channel)
Gas-sensor module near joints	MQ-series sensor detects combustible gas near pipe joints and triggers a buzzer	Selected (leak channel)
Single controller	ESP32 / Arduino compares both readings with set thresholds	Selected
LOW and CRITICAL states	Warn before the gas runs out, not after	Selected
Mobile notification	Alert when the level drops faster than normal use would explain	Optional, future stage
Browser dashboard	Simulate the full logic first to test the interface and alert behaviour	Selected (built prototype)

3.4 Prototype

The implemented prototype is **BioGuard_Demo.html**, called the BioGuard Console. It is a responsive web page with a gas-availability gauge, a 10-segment LED-style bar, virtual sensor channels, scenario buttons, a trend chart, an event log, editable thresholds and alarm controls. Its readings are generated locally by JavaScript, and it runs in a modern browser without installation or network access.

3.5 Test

The prototype was tested using normal use, low-gas, critical-gas, possible-leak, reset, threshold-change, quiet-mode and boundary scenarios. The tests validate the interface and the alert logic only. They do not validate a physical sensor or real gas detection. The test cases and results are given in Chapter 6.

CHAPTER 4: REQUIREMENTS

4.1 Functional Requirements

The system must show the available gas, warn about low and critical levels, warn about a possible leak and let the user control the thresholds and alarm. Table 4.1 lists the functional requirements, all of which are implemented in the prototype.

Table 4.1: *Functional requirements*

ID	Requirement	Implemented in
FR1	Display the available gas as a percentage index, with a gauge and a 10-segment LED-style bar	Gas index card, reserve gauge
FR2	Classify the level as GOOD, LOW or CRITICAL using editable thresholds	Gas-level engine
FR3	Display a virtual outlet-pressure value and a leak-sensor value	Pressure card, sensor channels
FR4	Raise a possible-leak alert when the reading reaches the leak threshold or a leak test is run	Leak monitor
FR5	Sound a pulse, rapid or steady alarm tone with adjustable volume and a mute option	Alarm engine
FR6	Provide alert modes: gas and leak, gas only, leak only, or quiet	Control room
FR7	Show a trend of the last 10 readings and an event log of the last 30 events	Trend chart, event log
FR8	Allow simulated gas use, a sample reading, reset, and saving or loading of settings	Action buttons

4.2 Non-Functional Requirements

The interface should be easy to understand for non-technical users, respond immediately to every change and work without errors. It should run offline on a normal computer or phone browser and remain readable on different screen sizes.

Table 4.2: *Non-functional requirements*

Attribute	Requirement
Usability	Large readable values, clear colour cues (green, amber, red) and simple buttons
Performance	Dashboard updates instantly when a slider, input or button changes
Reliability	No page errors during the tested scenarios; values are limited to valid ranges
Portability	Single HTML file; runs in Chrome or Edge with no installation or network
Responsiveness	Layout adapts from desktop to tablet and mobile widths
Safety and honesty	Every screen states that values are illustrative and not a real gas alarm

4.3 Software and Hardware Requirements

Software Requirements:

- HTML5, CSS3 and JavaScript (no external library)
- SVG for the gauge and trend chart, Web Audio API for alarm tones, localStorage for saved settings
- Web browser: Chrome or Edge (any modern browser)
- Code editor: VS Code

Hardware Requirements (to run the prototype):

- Computer, laptop or smartphone
- Minimum 4 GB RAM
- Speaker or headphones to hear the alarm tones

Proposed hardware for the physical version:

Table 4.3: *Proposed hardware modules (not connected in the current prototype)*

Module	Function	Channel
Low-pressure sensor	Estimates stored gas at the digester outlet	Level
MQ-series gas sensor	Detects combustible gas near pipe joints	Leak
ESP32 / Arduino	Reads sensors and compares with thresholds	Controller
LED bar-graph	Shows the gas level, readable from the kitchen	Level output
Buzzer	Sounds the leak alarm without needing a smartphone	Leak output
Wi-Fi (ESP32), optional	Mobile notification when the level drops faster than normal use	Optional alert

CHAPTER 5: SYSTEM DESIGN AND IMPLEMENTATION

5.1 System Architecture

Figure 5.1 shows the architecture of BioGuard. The upper part is the proposed physical system and the lower part is the browser simulation that has been implemented. Both follow the same flow: **Input → Processing → Threshold comparison → Alert → Display**.

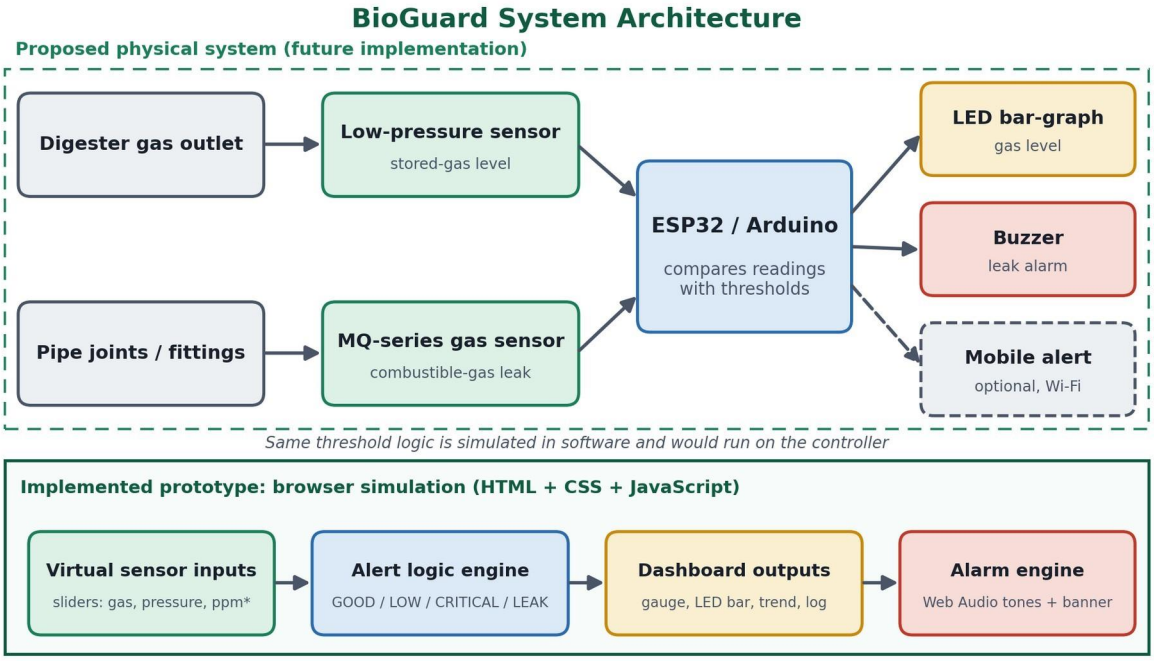


Figure 5.1: *System architecture of BioGuard*

In the physical design, the low-pressure sensor and the MQ-series sensor feed an ESP32 or Arduino controller. The controller compares the readings with thresholds, drives the LED bar-graph for the level and the buzzer for the leak alarm, and can optionally send a mobile alert. In the implemented prototype, sliders and buttons play the role of the sensors, a JavaScript engine applies the same threshold logic and the dashboard and Web Audio alarm act as the outputs.

5.2 Alert Decision Logic

The alert logic runs two independent channels, as shown in Figure 5.2. The gas-level channel classifies the index as GOOD, LOW or CRITICAL. The leak channel raises an alert when the leak reading reaches the threshold or when a leak test is active. Both results update the gauge, LED bar, trend and event log, and the alarm tone starts for a leak or a critical level unless the system is muted or in quiet mode.

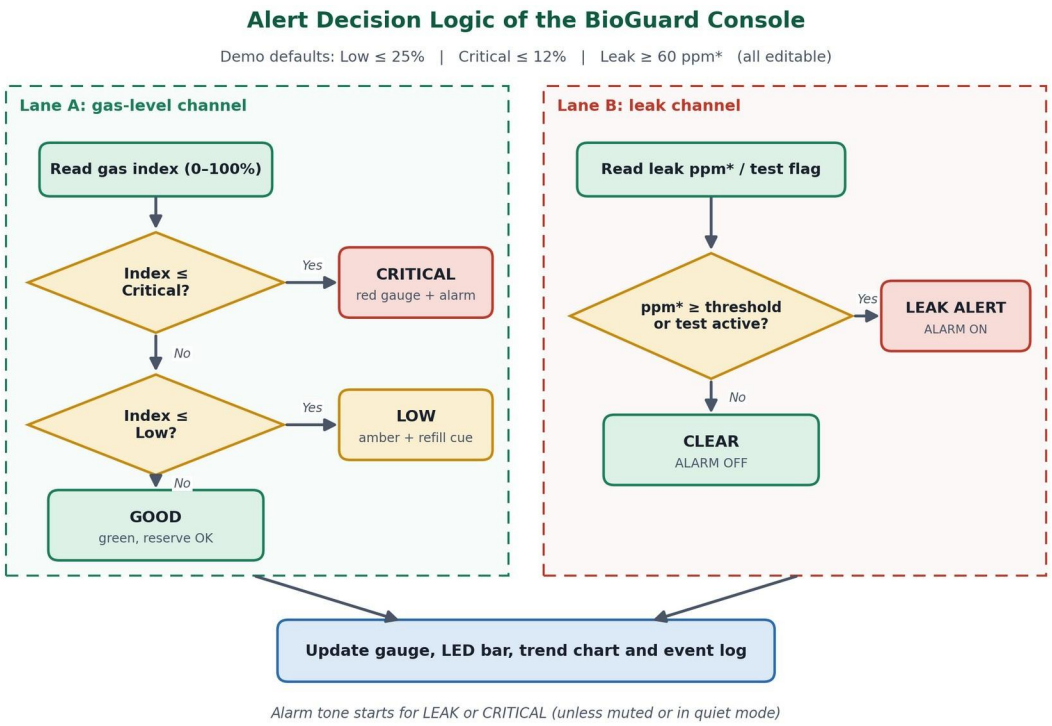


Figure 5.2: *Alert decision logic of the BioGuard Console*

Table 5.1: *Alert states and conditions (demo defaults)*

State	Condition	Visual cue	Alarm tone
GOOD	Index above 25%	Green gauge; "Comfortable reserve"; system READY	None
LOW	Index 25% or below (above 12%)	Amber gauge and LED segments; "Refill / conserve gas soon"; system LOW GAS	None (visual only)
CRITICAL	Index 12% or below	Red gauge; "Reserve critically low"; system CRITICAL	Yes
LEAK ALERT	Leak reading 60 ppm* or above, or leak test active	Leak monitor ALERT, ALARM ON, red flashing edge and banner	Yes

The alert behaviour setting changes which channels are active: "Gas + leak alerts" (default), "Gas alerts only", "Leak alerts only" and "Dashboard only (quiet)". All thresholds are editable: the low threshold between 5 and 60%, the critical threshold between 1 and 40% (always below the low threshold) and the leak threshold between 10 and 100 ppm*. *Units marked with an asterisk are illustrative.

5.3 GUI Design

The BioGuard Console uses a dark dashboard layout with large, readable values and colour-coded states. The home view in Figure 5.3 shows the header with the live simulation badge and clock, the action buttons (Simulate gas use, Test leak alert, Reset demo, Theme, Mute alarm) and four status cards: available gas index with the 10-segment LED bar, outlet pressure, leak monitor and system state.

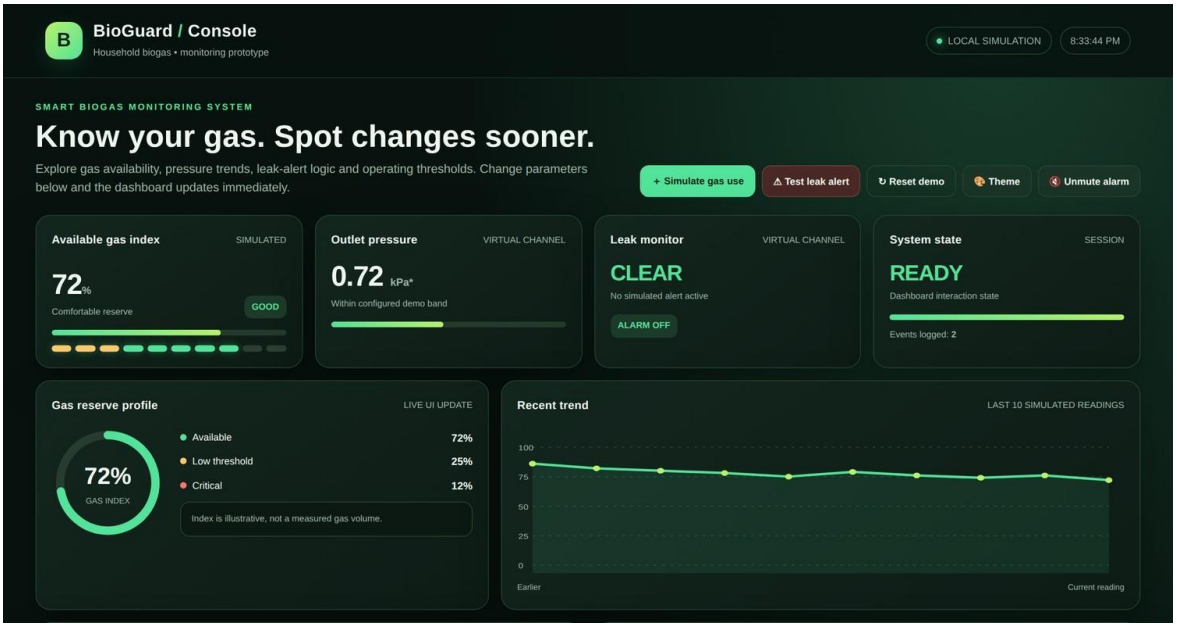


Figure 5.3: *BioGuard Console home view in normal state (gas 72% GOOD, leak CLEAR)*

Below the cards, the gas reserve profile shows a circular gauge with the available, low and critical values, and the recent trend plots the last 10 simulated readings (Figure 5.4).



Figure 5.4: *Gas reserve profile and recent trend of the last 10 readings*

The control room holds the editable parameters: gas availability index, leak sensor reading, virtual pressure, gas use per action, the three thresholds and the alert behaviour. Changes apply instantly. The alarm and safety panel provides Test sound, Stop alarm and Mute, a volume slider and three sound patterns (Figure 5.5).

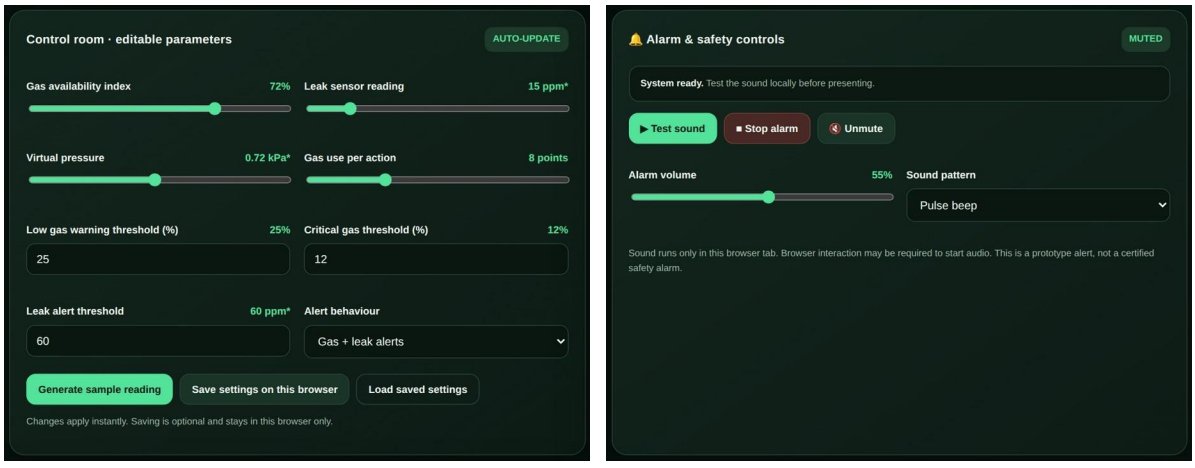


Figure 5.5: *Control room (editable parameters) and alarm and safety controls*

The sensor channels panel lists the three illustrative channels (gas availability, outlet pressure and gas sensor) together with a prototype-boundary notice, and the event log records every action with a time stamp (Figures 5.6 and 5.7).

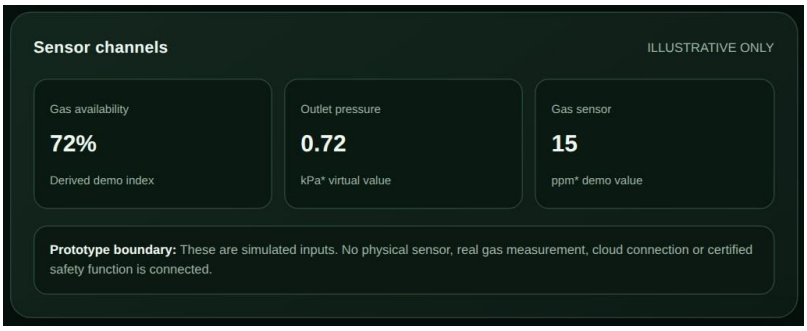


Figure 5.6: *Sensor channels (illustrative only) with prototype-boundary notice*

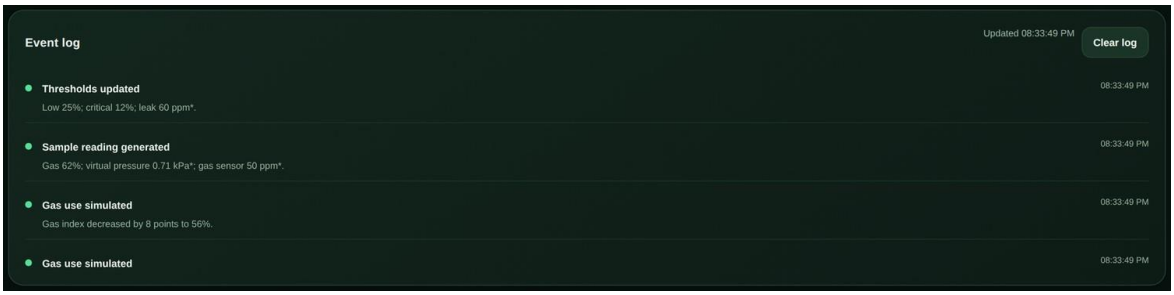


Figure 5.7: *Event log of recent actions*

Table 5.2: *Dashboard components*

Component	What it shows	Updated by
Available gas index	Percentage, state badge, 10-segment LED bar	Gas slider, gas use, reset
Outlet pressure	Virtual pressure in kPa* and a band message	Pressure slider
Leak monitor	CLEAR or ALERT, ALARM OFF or ALARM ON	Leak slider, leak test
System state	READY, LOW GAS, CRITICAL or ALERT, event count	Combined logic
Gas reserve profile	Circular gauge with available, low and critical values	Gas index, thresholds
Recent trend	Line chart of the last 10 readings	Gas changes
Control room	Sliders, number inputs and alert behaviour	User input
Alarm and safety	Test, stop, mute, volume and sound pattern	User input, alerts
Event log	Latest 30 events with time stamp	All actions

5.4 Data Design

BioGuard has no server or database. All data is held in memory as a single JavaScript state object and is redrawn on every change. The only persistent storage is the optional “Save settings

on this browser" action, which writes the parameter values to localStorage under the key bioguardSettings. Table 5.3 lists the main state variables.

Table 5.3: *Main state variables of the console*

Variable	Default	Meaning
gas	72	Available gas index (0 to 100%)
leak	15	Virtual gas-sensor reading in ppm* (0 to 100)
pressure	72	Virtual pressure; shown as 0.72 kPa* (value divided by 100)
usage	8	Gas index points used per "Simulate gas use" action (1 to 25)
low / critical	25 / 12	Low-gas and critical-gas thresholds in %
leakThreshold	60	Leak alert threshold in ppm*
alarmMode	both	both, gas, leak or quiet
leakActive	false	True while a leak test is active
history	10 values	Last 10 gas readings for the trend chart
events	≤ 30 items	Event log entries with time, title, detail and alert flag
muted / volume / soundType	false / 55 / pulse	Alarm settings

5.5 Implementation Details

The console is implemented with HTML for structure, CSS for the responsive layout and themes, and JavaScript for the logic. Event handlers on the sliders, inputs and buttons update the state object and call a single refresh() function that redraws every part of the dashboard. Figure 5.8 shows the six software modules.

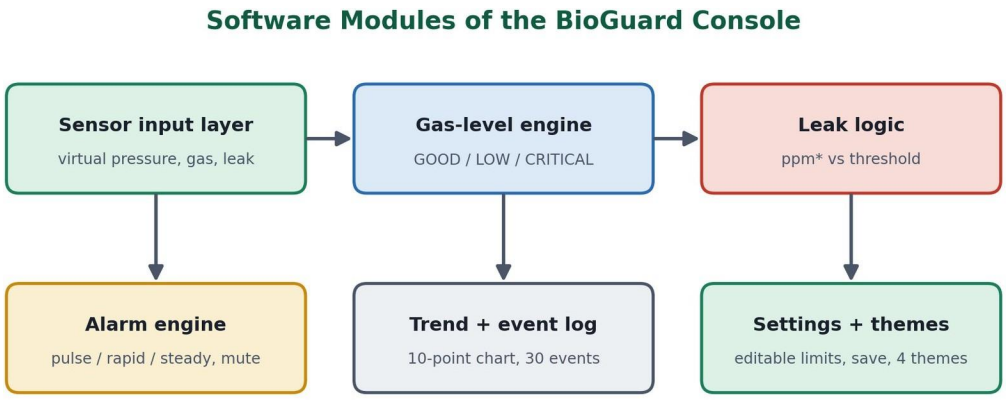


Figure 5.8: *Software modules of the BioGuard Console*

Table 5.4: *Key functions of the prototype*

Function	Purpose
refresh()	Reads the state, classifies gas and leak status, and updates cards, gauge, LED bar, system state and alarm
drawChart()	Draws the 10-point trend as an SVG polyline with a grid and markers
renderEvents() / addEvent()	Maintains the event log (latest 30 entries, newest first)
simulateUse()	Reduces the gas index by the configured number of points (never below 0)
triggerLeak()	Activates a simulated leak alert and logs the event
applyRandomReading()	Generates a sample gas, pressure and leak reading
resetDemo()	Restores all default values and the initial trend
setThresholds()	Validates and applies the low, critical and leak thresholds
saveSettings() / restoreSettings()	Stores and loads parameter values in the browser (localStorage)
syncAlarm() / beep() / startTone()	Shows the alert banner and plays pulse, rapid or steady tones using the Web Audio API
toggleTheme() / toggleAlarmMute()	Cycles four themes (forest green, ocean, violet, light) and mutes or unmutes the alarm

CHAPTER 6: TESTING AND RESULTS

6.1 Testing Methodology

Testing ensures that the dashboard behaves correctly and that the alert logic matches the design. The prototype was checked using the following techniques:

- **Scenario testing:** normal use, low gas, critical gas, possible leak and reset.
- **Input-output testing:** sliders, number inputs and buttons were set to defined values and the displayed outputs were compared with the expected results.
- **Boundary testing:** repeated gas use down to 0% to confirm that the index never becomes negative.
- **Configuration testing:** changed thresholds and alert behaviour, including quiet mode.
- **Alarm control testing:** stop alarm and mute behaviour.

The tests were run in a Chromium-based browser by operating the page controls and reading the values shown on the dashboard. No page errors were reported during the run. These are software interface tests only; no physical sensor, gas or hardware test has been performed, and none should be attempted by releasing gas.

6.2 Test Cases and Results

Table 6.1 lists the twelve test cases that were executed, with the expected and observed outputs.

Table 6.1: *Test cases and results*

No.	Test scenario / input	Expected output	Actual output	Result
1	Open the page (initial state)	72% GOOD, leak CLEAR, ALARM OFF, state READY	72% GOOD, 8 LED segments, CLEAR, ALARM OFF, READY	**Pass**
2	Simulate gas use once (8 points)	Index falls to 64%; event logged	64% GOOD, 7 LED segments; event added	**Pass**
3	Set gas index to 24%	LOW state with refill message	LOW, “Refill / conserve gas soon”, system LOW GAS	**Pass**
4	Set gas index to 9%	CRITICAL state and alert banner	CRITICAL, “Reserve critically low”, alert banner active	**Pass**
5	Press Test leak alert	Leak ALERT and ALARM ON	ALERT, ALARM ON, system ALERT, event logged	**Pass**
6	Press Stop alarm while the leak reading is still at the threshold	Stop event logged; alert re-activates because the threshold is still crossed (by design)	Event logged; ALERT stayed active (reading 60 vs threshold 60) until the demo was reset	**Pass**
7	Press Reset demo	All defaults restored	72% GOOD, CLEAR, READY, alarm off	**Pass**
8	Set leak reading to 65 ppm* (threshold 60)	Leak ALERT	ALERT, ALARM ON	**Pass**

No.	Test scenario / input	Expected output	Actual output	Result
9	Select "Dashboard only (quiet)" with 65 ppm*	No alert in quiet mode	CLEAR, ALARM OFF, READY	Pass
10	Change low threshold to 30% and set gas to 28%	LOW state with the new threshold	LOW, system LOW GAS	Pass
11	Simulate gas use 10 times from 72%	Index stops at 0%, never negative	0% CRITICAL; trend ends 8, 0, 0	Pass
12	Press Mute alarm	Sound status shows MUTED; visual alerts remain	Status MUTED; banner and states still shown	Pass

All twelve test cases produced the expected output. The dashboard responded immediately to every input, and the gauge, LED bar, state labels, trend chart and event log stayed consistent with each other.

6.3 Output Screens

The following screens show the main output states of the prototype. In normal operation (Figure 6.1) the reserve is comfortable and no alert is active.

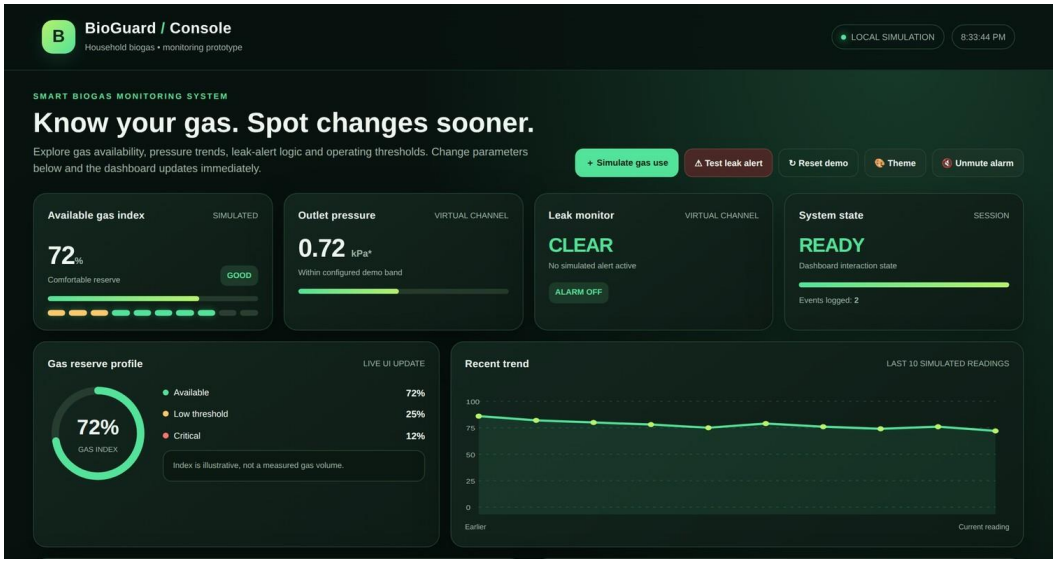


Figure 6.1: *Output screen: normal operation (gas 72% GOOD, leak CLEAR)*

When the gas index falls to the low threshold (Figure 6.2) the status turns amber and the reserve is described as needing a refill or conservation.

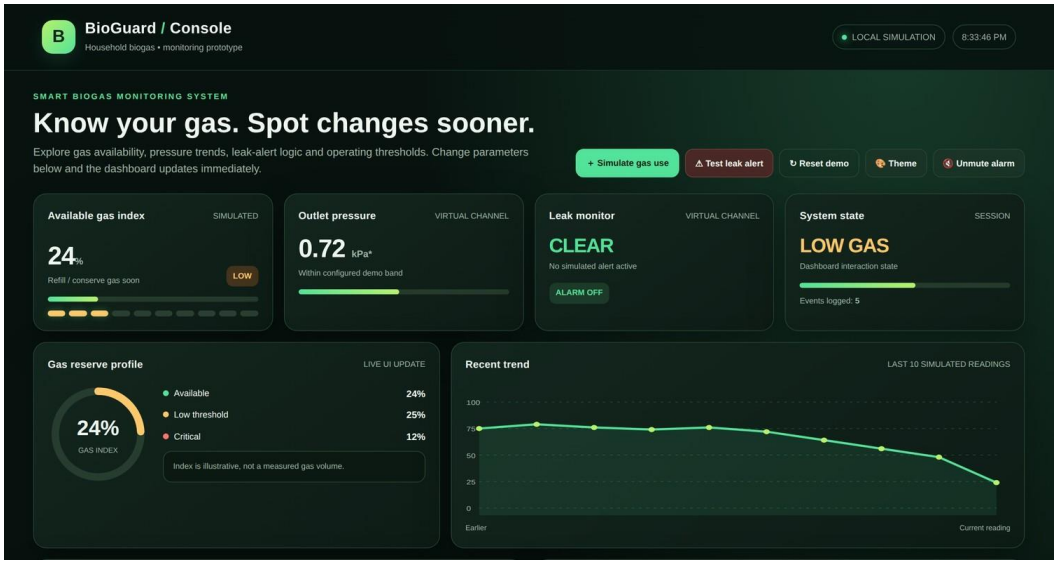


Figure 6.2: *Output screen: low-gas warning (gas 24% LOW, amber cue, system LOW GAS)*

At or below the critical threshold (Figure 6.3) the gauge and state turn red and the alert banner and alarm tone become active.

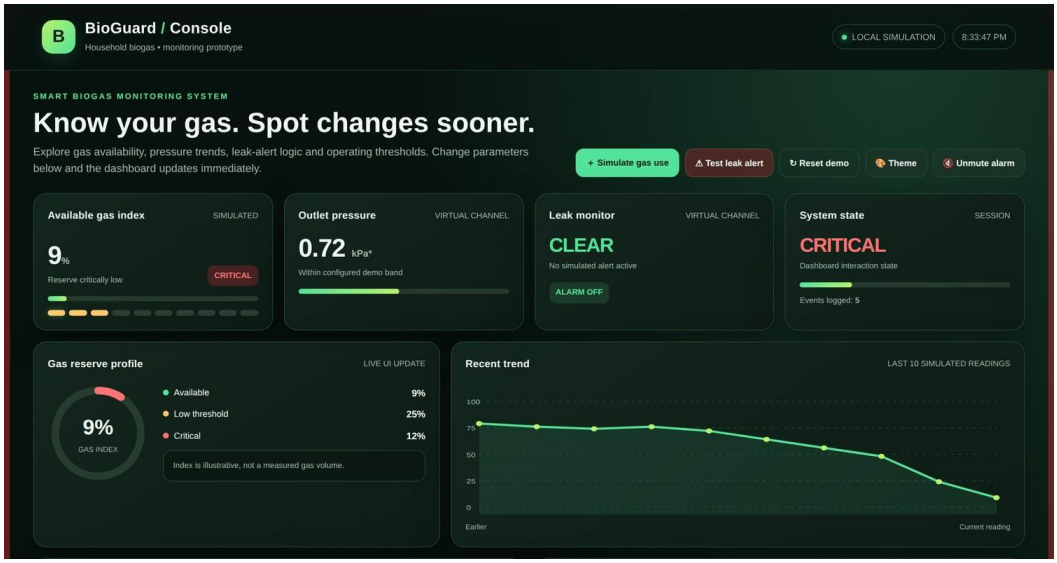


Figure 6.3: *Output screen: critical gas level (gas 9% CRITICAL, red state, alert active)*

When the leak reading reaches the threshold or the leak test is pressed (Figure 6.4), the leak monitor shows ALERT with ALARM ON and the page border flashes red. The sound itself is muted in the captured image.

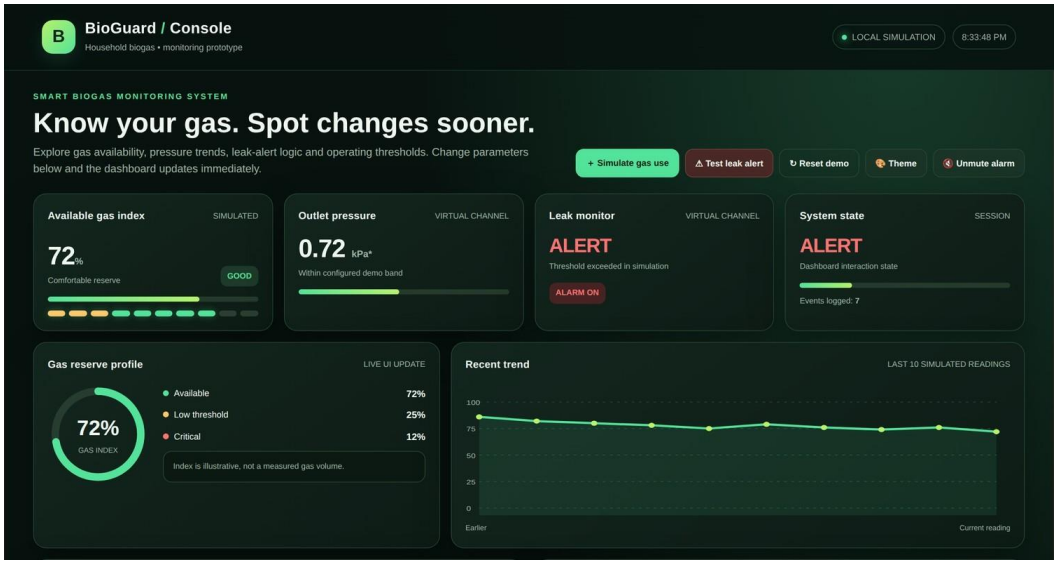


Figure 6.4: *Output screen: possible-leak alert (leak ALERT, alarm ON, system ALERT)*

Table 6.2: *Summary of output states*

Scenario	Gas index	Gas state	Leak monitor	Alarm	System state
Normal operation	72%	GOOD	CLEAR	OFF	READY
Low gas	24%	LOW	CLEAR	OFF	LOW GAS
Critical gas	9%	CRITICAL	CLEAR	Banner and tone	CRITICAL
Possible leak	72%	GOOD	ALERT	ON	ALERT

The results confirm that the prototype shows the three gas states, raises a leak alert and handles reset, configuration and alarm controls as designed. The values are simulated, so the results show the behaviour of the interface and logic and not the accuracy of any sensor.

CHAPTER 7: FEASIBILITY, IMPACT AND SAFETY

7.1 Feasibility Analysis

The feasibility of the idea was analysed from the technical, economic and operational points of view (Table 7.1). The overall verdict is that BioGuard is feasible as a bench-tested prototype with standard precautions.

Table 7.1: *Feasibility analysis*

Aspect	Assessment	Basis
Technical	Common sensor modules and controllers can implement the level and leak channels and can be tested on a bench	Pressure sensor, MQ-series sensor and ESP32 / Arduino are widely used modules
Economic	Low-cost sensors, one controller and a basic display; actual cost depends on the chosen components and local prices	Basic LED bar and buzzer; no cloud service required
Operational	LED bar and buzzer give a local indication in the kitchen without internet or a smartphone	Editable thresholds and alert modes adapt to the household

Verdict: feasible as a bench-tested prototype with standard precautions.

7.2 Impact and Benefits

MNRE reports about 4.31 million family-type biogas plants set up in India. The target users are rural and semi-urban homes cooking with 1 to 6 m3 family plants. BioGuard answers the question “how much gas is left?” and gives an early warning before the flame weakens or a leak grows. Table 7.2 summarises the potential benefits of a validated physical system; the current software prototype demonstrates the interface and logic only.

Table 7.2: *Potential impact and benefits*

Area	Potential benefit
Social	Safer kitchens and more trust in biogas adoption
Economic	Less wasted fuel and fewer cooking interruptions
Environmental	Keeps clean cooking fuel in use and flags escaping methane
Safety	Early warning of combustible gas near pipe joints

7.3 Safety Considerations

Never deliberately release biogas or create a gas leak to test this project. Any future hardware testing must use safe, controlled procedures and appropriate supervision. A dashboard warning cannot establish that a location is safe and is not a substitute for appropriate emergency procedures or qualified advice. The BioGuard Console is a design-thinking demonstration, not a certified gas-safety device, and must not be used for real-world gas-leak detection or safety decisions.

CHAPTER 8: CONCLUSION AND FUTURE WORK

8.1 Limitations

The current work has the following limitations:

- It is a software simulation only; no physical sensor, ESP32 / Arduino, LED bar or buzzer is connected.
- The ppm* and kPa* units are illustrative and not calibrated.
- The gas index is not a measured gas volume; a physical system needs calibration for the specific digester.
- It is not a certified gas-safety device.
- There is no mobile or cloud alert yet; alerts stay in the browser tab.
- Browser limits apply: audio may need a click to start, and saved settings are stored per browser.

8.2 Conclusion

BioGuard successfully demonstrates, as a browser-based simulation, how a household biogas monitor could present gas availability and warn about a possible leak. The console shows GOOD, LOW and CRITICAL gas states, a possible-leak alert, a trend chart, an event log, editable thresholds and configurable alarm behaviour. All twelve test cases matched the expected logic.

Through this project, the design-thinking stages of empathizing, defining, ideating, prototyping and testing were applied to a real household problem, and web-development concepts such as event handling, dynamic content generation, SVG graphics, audio generation and interface design were applied in the prototype. The work is a software prototype rather than a completed hardware safety system; sensor integration, calibration and controlled validation remain future work.

8.3 Future Work

The roadmap towards a physical prototype is:

1. Integrate a compatible low-pressure sensor and an MQ-series sensor with an ESP32 or Arduino controller.
2. Calibrate the pressure-to-level relationship and the leak threshold for a particular digester.
3. Connect a physical LED bar-graph and buzzer to the controller.
4. Investigate abnormal level drops compared with expected normal use and add a mobile alert only after reliable measurement and connectivity are established.
5. Validate the system safely, under supervision, in the field.

In conclusion, this project serves as a foundation for a low-cost household biogas level and leak monitor and can be extended significantly once physical sensing and validation are added.

APPENDIX

Source Code (selected excerpts)

The complete prototype is a single file, BioGuard_Demo.html, containing the HTML structure, CSS styles and JavaScript logic. The main parts of the logic are reproduced below.

A.1 State object (BioGuard_Demo.html)

```
const state = {
  gas: 72, leak: 15, pressure: 72, usage: 8,
  low: 25, critical: 12, leakThreshold: 60,
  alarmMode: 'both', leakActive: false,
  events: [], history: [86,82,80,78,75,79,76,74,76,72],
  muted: false, alarmPlaying: false, volume: 55,
  soundType: 'pulse', audioCtx: null, alarmTimer: null, themeIndex: 0
};
```

A.2 Gas-use simulation

```
function simulateUse() {
  state.gas = Math.max(0, state.gas - state.usage);
  state.history.push(state.gas);
  state.history = state.history.slice(-10);
  addEvent('Gas use simulated',
    `Gas index decreased by ${state.usage} points to ${state.gas}%.`,
    state.gas <= state.low);
  refresh();
}
```

A.3 Leak-alert test

```
function triggerLeak() {
  state.leakActive = true;
  state.leak = Math.max(state.leak, state.leakThreshold);
  addEvent('Leak alert test',
    'Simulated leak alert activated. This is not a real sensor reading.',
    true);
  refresh();
}
```

A.4 State classification inside refresh() (formatted for readability)

```
let gasWord = state.gas <= state.critical ? 'CRITICAL'
            : state.gas <= state.low      ? 'LOW' : 'GOOD';

let leakOn = state.leakActive ||
  (state.alarmMode !== 'gas' && state.alarmMode !== 'quiet'
   && state.leak >= state.leakThreshold);
if (state.alarmMode === 'quiet') leakOn = false;

let gasAlert = state.gas <= state.low
  && state.alarmMode !== 'leak' && state.alarmMode !== 'quiet';
let critical = state.gas <= state.critical;

syncAlarm(leakOn || (critical && state.alarmMode !== 'quiet'
                             && state.alarmMode !== 'leak'));
```

A.5 Reset to default demo state

```
function resetDemo() {
  state.gas = 72; state.leak = 15; state.pressure = 72; state.usage = 8;
  state.low = 25; state.critical = 12; state.leakThreshold = 60;
  state.leakActive = false;
  state.history = [86,82,80,78,75,79,76,74,76,72];
  $('modeSelect').value = 'both';
  initInputs();
  addEvent('Demo reset', 'Illustrative readings restored to defaults.');
  refresh();
}
```

How to run the prototype: open BioGuard_Demo.html in Chrome or Edge. Use Simulate gas use, Test leak alert, Reset demo and the control-room sliders to explore the scenarios. Use Test sound after a click if the browser requires interaction before playing audio.

Description: The BioGuard Console is developed using HTML, CSS and JavaScript. HTML and CSS create the dashboard, JavaScript handles user interaction, alert logic, the trend chart and the alarm tones, and no backend is required. All readings are illustrative.

References

1. Cross-Domain Design Thinking Problem Statement Compendium, “Low-Cost Biogas Level and Leak Monitor for Household Digesters,” Problem Statement 24, p. 25. Domain: Bioenergy / Environmental Biotechnology.
2. Hanwei Electronics, “MQ-4 Semiconductor Sensor for Natural Gas,” datasheet: sparkfun.com/datasheets/Sensors/Biometric/MQ-4.pdf
3. Ministry of New and Renewable Energy (MNRE), Government of India, “Biogas FAQs”: mnre.gov.in/faq
4. Stanford d.school, “Design Thinking Bootleg”: dschool.stanford.edu/resources/design-thinking-bootleg
5. Espressif Systems, “ESP32 Documentation”: docs.espressif.com
6. MDN Web Docs, HTML, CSS and JavaScript documentation: https://developer.mozilla.org/en-US/
7. MDN Web Docs, Web Audio API: https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
8. MDN Web Docs, Window.localStorage: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
9. BioGuard demo (this project): browser-based simulation of level, leak and alarm logic, BioGuard_Demo.html.